



**INSTITUTO DE EDUCACIÓN SECUNDARIA LA MOLA
NOVELDA**

DESARROLLO DE APLICACIONES WEB

Curso Académico 2025/2026

Desarrollo Web en Entorno Cliente

**U04_A03 Casos prácticos, Ponte a prueba, etc... -
Prof. Emilio Ramírez**

Autor: Segura Abad, Mario

Fecha: 18 de Enero, 2025

ÍNDICE

INTRODUCCIÓN.....3

ENUNCIADO.....4

Probad todo el código del Apartado 3.....4

Caso práctico 34

Procesamiento de múltiples promesas en paralelo con *async/await*.....4

Ponte a prueba 3.....4

Conversión de promesas a *async/await*4

 Claves de resolución:.....4

DESARROLLO DE LA PRACTICA.....5

Descripción5

 • Ejecutar múltiples promesas en paralelo y esperar a todas ellas con `await` (Caso práctico 3).5

 • Convertir código basado en `.then()` a sintaxis `async/await` (Ponte a prueba 3).5

Contenido del archivo5

Código del Apartado 3 de teoría.....5

Caso práctico 3 – Procesamiento de múltiples promesas en paralelo con *async/await*7

Ponte a Prueba 3 – Conversión de promesas a *async/await*.....8

Cómo visualizarlo.....9

Requisitos:9

Pasos:9

Objetivos de la práctica9

 • Probar y comprender el código teórico del apartado 3.9

 • Ejecutar múltiples promesas en paralelo usando `async/await`.9

 • Convertir código basado en `.then()` a `async/await`.9

CONCLUSIÓN10

INTRODUCCIÓN

En el ejercicio “Probad todo el código del Apartado 3” deberemos incluir en uno o varios fuentes (yo siempre recomiendo varios) absolutamente todo el código que aparece en la teoría del apartado 3 de Programación asíncrona, así como su ejecución y prueba de autoría

En el “Caso práctico 3” crearemos un programa basado en promesas lanzando tres temporizadores en paralelo. Una vez finalicen, se deberá mostrar en consola los valores devueltos.

En el “Ponte a prueba 3” tendremos que transformar el código del apartado anterior, el 2 para utilizar `async/await` en vez de promesas. Disponemos para ello, del archivo `DWEC_U04_A02_01.html`.

ENUNCIADO

Probad todo el código del Apartado 3

Incluid en uno o varios fuentes **todo** el código que aparece en la teoría de este apartado, así como su ejecución y prueba de autoría.

Caso práctico 3

Procesamiento de múltiples promesas en paralelo con *async/await*

Crea un programa basado en promesas que lance **tres temporizadores con distintos tiempos en paralelo**. A continuación, el programa debe **esperar** utilizando **await** a que **finalicen los tres**. Una vez que hayan finalizado, el programa deberá mostrar en la consola los valores que devuelvan.

Ponte a prueba 3

Conversión de promesas a *async/await*

Transforma el código del ejemplo estudiando en la sección *Un ejemplo: API Media Devices* del Apartado 2, **DWEC_U04_A02_01.html**, para que utilice **async/await** en lugar de promesas.

Claves de resolución:

- Recuerda que solo puedes utilizar **await** dentro de módulos y de funciones definidas como **async**.
- **await** espera a que se resuelva la promesa y devuelve directamente el resultado.

DESARROLLO DE LA PRACTICA

Práctica DWEK – Unidad 4 Apartado 3

Descripción

Este apartado se centra en el uso de **async/await** en JavaScript para gestionar operaciones asíncronas de forma más clara y legible.

El objetivo principal ha sido:

- Ejecutar múltiples promesas en paralelo y esperar a todas ellas con `await` (Caso práctico 3).
- Convertir código basado en `.then()` a sintaxis `async/await` (Ponte a prueba 3).

Contenido del archivo

Código del Apartado 3 de teoría

Se ha probado todo el código relacionado con:

- Uso de funciones `async` y la palabra clave `await`.
- Ejecución de varias promesas simultáneamente con `Promise.all()`.
- Manejo de errores con `try/catch`.
- Conversión de código basado en promesas a código `async/await`.

Cada archivo incluye comentarios con el nombre del autor y la fecha de realización.

Se han capturado los resultados por consola como prueba de ejecución.

Ejemplo de ejecución:

Salida por consola – DWEK_U04_A03_TEO_1.js

```
▼ Promise {<fulfilled>: 15} ⓘ  
  ► [[Prototype]]: Promise  
    [[PromiseState]]: "fulfilled"  
    [[PromiseResult]]: 15  
    
  Autor: Mario Segura Abad  
  Fecha de realización: 18/01/2026  
  15  
  undefined
```

Salida por consola – DWEC_U04_A03_TEO_2.js

```
console.log("Fecha de realización: 18/01/2026");  
Autor: Mario Segura Abad  
Fecha de realización: 18/01/2026  
< undefined  
Temporizador de 3000 ms terminado
```

Salida por consola – DWEC_U04_A03_TEO_3.js

```
// Prueba de Autoría  
console.log("Autor: Mario Segura Abad");  
console.log("Fecha de realización: 16/01/2026");  
✖ ▶ Uncaught ReferenceError: miFuncionAsincrona is not defined  
   at <anonymous>:8:1
```

Salida por consola – DWEC_U04_A03_TEO_4.js

```
Autor: Mario Segura Abad  
Fecha de realización: 18/01/2026  
< undefined
```

Salida por consola – DWEC_U04_A03_TEO_5.js

```
Autor: Mario Segura Abad  
Fecha de realización: 18/01/2026  
< undefined  
vuejs
```

Salida por consola – DWEK_U04_A03_TEO_6.js

```
Autor: Mario Segura Abad
Fecha de realización: 18/01/2026
< undefined
vuejs
```

Salida por consola – DWEK_U04_A03_TEO_7.js

```
Autor: Mario Segura Abad
Fecha de realización: 18/01/2026
< undefined
vuejs
```

Caso práctico 3 – Procesamiento de múltiples promesas en paralelo con async/await**Objetivo:**

- Crear un programa que lance **tres temporizadores en paralelo**, cada uno con un tiempo distinto.
- Esperar a que finalicen los tres usando `await Promise.all()`.
- Mostrar en consola los valores devueltos por cada temporizador.

Claves de resolución:

- Crear una función `temporizador(t)` que devuelva una promesa que resuelve tras `t` milisegundos.
- Lanzar las promesas simultáneamente.
- Esperar a todas.
- Mostrar los resultados en consola.

Este ejercicio ha reforzado el uso de **async/await** para gestionar tareas asíncronas en paralelo de forma clara y eficiente.

Ejemplo de ejecución:

Salida por consola – CasoPractico3.js

```
Autor: Mario Segura Abad
Fecha de realización: 18/01/2026
< undefined
Temporizador de 1000 ms terminado
Temporizador de 2000 ms terminado
Temporizador de 3000 ms terminado
```

Ponte a Prueba 3 – Conversión de promesas a async/await

Objetivo:

- Transformar el código del ejemplo de *Media Devices* (que usa `.then()` y `.catch()`) para que utilice `async/await`.

Claves de resolución:

- Recordar que `await` solo puede usarse dentro de funciones `async` o módulos ES.
- `await` **pausa** la ejecución hasta que la promesa se resuelve.
- El resultado se obtiene directamente sin necesidad de `.then()`.

Ejemplo de ejecución:

Salida por consola – DWEK_U04_A03_01.html



Autor: **Mario Segura Abad**

Fecha de realización: 18/01/2026

Cómo visualizarlo

Requisitos:

- Editor de código compatible: en mi caso, Visual Studio Code.
- Navegador web actualizado: en mi caso, Google Chrome.
- (Opcional) Node.js para ejecutar los scripts en consola.

Pasos:

1. Guardar los archivos .js y .html en la raíz del proyecto, **muy importante este paso.**
2. Si se usa await fuera de funciones, enlazar el script como módulo:


```
<script type="module" src="archivo.js"></script>
```
3. Abrir el archivo .html en el navegador o ejecutar el .js con Node.
4. **Caso práctico 3:** observar cómo los tres temporizadores se ejecutan en paralelo y devuelven sus resultados juntos.
5. **Ponte a prueba 3:** comprobar que el código convertido a async/await funciona correctamente.
6. Revisar la consola (F12 → "Consola") para verificar mensajes de prueba de autoría.

Objetivos de la práctica

- Probar y comprender el código teórico del apartado 3.
- Ejecutar múltiples promesas en paralelo usando async/await.
- Convertir código basado en .then() a async/await.
- Documentar correctamente cada ejercicio con nombre, fecha y capturas de pantalla.
- Presentar el proyecto en formato PDF junto con los archivos fuente y pantallazos.

CONCLUSIÓN

Esta práctica nos ha permitido consolidar el uso de **async/await** en JavaScript para gestionar procesos asíncronos de forma más clara y estructurada.

Se ha comprobado cómo ejecutar varias tareas en paralelo, cómo capturar errores y cómo transformar código basado en promesas en código más legible.

La práctica, asimismo, nos ha sido muy útil para entender la programación asíncrona en aplicaciones web.